



NODE JS

Curso: Desarrollo en entorno servidor con Node.js UF1844

INTRODUCCIÓN	1
MÓDULOS EN NODE.JS	2
Módulos Propios.	3
Módulos nativos fundamentales.	5
📁 fs (File System)	5
🌐 path	7
🌐 http	9
Fetch(): Comunicación entre el Frontend y el Servidor	16
El paquete NPM.	21

INTRODUCCIÓN

[Node](#) (o más correctamente: **Node.js**) es un entorno que trabaja en tiempo de ejecución, de código abierto, multi-plataforma, que permite a los desarrolladores crear toda clase de herramientas de **lado servidor** y aplicaciones en [JavaScript](#). La ejecución en tiempo real está pensada para usarse fuera del contexto de un explorador web (es decir, ejecutarse directamente en una computadora o sistema operativo de servidor).

Node.js es un entorno de ejecución de JavaScript que funciona fuera del navegador. Con Node, el **mismo lenguaje (JS)** se usa tanto en el **frontend** como en el **backend**. Esto simplifica el aprendizaje y permite compartir funciones o estructuras entre ambas partes.

Es de **código abierto y multiplataforma**. Las aplicaciones Node.js están escritas en JavaScript puro y se pueden ejecutar dentro del entorno Node.js en Windows, Linux, etc.

La característica más importante de NodeJS, y que ahora otra serie de lenguajes están aplicando, es la de **no ser bloqueante**. Es decir, si durante la ejecución de un programa hay partes que necesitan un tiempo para producirse la respuesta, NodeJS no detiene el hilo de ejecución del programa, esperando que esa parte acabe, sino que continúa procesando las siguientes instrucciones. Cuando el proceso lento termina, entonces realiza las instrucciones que fueran definidas para realizar con los resultados recibidos. NodeJS fue pensado desde el primer momento para potenciar los beneficios de la programación asíncrona.

Node incluye **NPM (Node Package Manager)**, un gestor de paquetes con millones de módulos reutilizables. Esto acelera el desarrollo: en lugar de escribir todo desde cero, se instalan librerías que ya resuelven tareas comunes (como Express, bcrypt, dotenv, etc.).

Desde una perspectiva de desarrollo de servidor web, Node tiene un gran número de ventajas:



MÓDULOS EN NODE.JS

Node tiene una extensa colección de módulos disponibles para la creación de programas, que resuelven la más variada gama de necesidades de desarrollo además también permite que los creamos nosotros como en React los componentes. Un **módulo** en Node.js es cualquier archivo .js que contiene funciones, objetos o clases reutilizables. Es como un componente en React, pero en el backend. Estos módulos hay que importarlos.

[Node.js](#) tiene dos formas de importar módulos.

La **forma tradicional**: con `require ()`. Este es el sistema tradicional de Node.js, usado durante muchos años. Funciona sin necesidad de configurar nada. Hoy en día está en desuso excepto en proyectos más antiguos o de prueba. El uso de `require()` para importar módulos en Node.js, asociado al sistema CommonJS, se considera la forma tradicional o "antigua" y se está desaconsejando a favor de la sintaxis `import/export` del sistema ECMAScript Modules (ESM).

En este curso usaremos la forma moderna a través de la sintaxis import...from. Esta es la forma moderna de importar, igual que en JavaScript del navegador (ES6). Permite escribir código más limpio y usar export e import directamente. Para usar este modelo hay que hacer un cambio en el package.json o crearlo si aún no lo tienes. Una vez hecho este cambio la sintaxis require() deja de funcionar. Tenlo en cuenta.

```
api > {} package.json > ...  
1  {  
2    "type": "module"  
3  }  
4
```

¿Qué tipos de modules existen en [node.js](https://nodejs.org/)? Tenemos que distinguir tres tipos.

Tipos de módulos en Node.js

Tipo de módulo	Descripción
◆ Nativo	Incluido en <u>Node.js</u> (no necesitas instalar nada). Ej: <code>fs</code> , <code>http</code> , <code>path</code> .
◆ De terceros	Instalados con <code>npm</code> . Ej: <code>express</code> , <code>axios</code> , <code>chalk</code> .
◆ Propio	Archivos <code>.js</code> que tú creas con funciones o clases reutilizables.

Módulos Propios.

Un módulo propio (también llamado módulo local) es un archivo JavaScript creado por ti mismo que contiene código reutilizable (funciones, objetos, clases, variables...) y que puedes importar en otros archivos de tu proyecto.

Vamos a crear un módulo propio.

Vamos a crear un módulo que se va a llamar sumar: una función que recibe dos parámetros y los devuelve. Luego vamos a llamar a ese módulo en otro archivo que lo vamos a llamar [sumando.js](#) y llamaremos a la función.

Aquí tienes el ejemplo.

```
JS sumar.js  X  JS sumando.js  JS promises.js

utils > JS sumar.js >  sumar
1  export function sumar(a, b) {
2    return a + b;
3  }
```

Este es nuestro módulo. Ahora lo importamos.

```
JS sumar.js  JS sumando.js  X  JS promises.js

JS sumando.js > ...
1  import { sumar } from './utils/sumar.js';
2  const resultado = sumar(4, 7);
3  console.log('Resultado de la suma:', resultado);
4
```

Ejercicio.

Ahora crea tu un módulo para restar y ejecuta la función pasándole dos números.

Módulos nativos fundamentales.

Vamos a ver los módulos nativos de [Node.js](#). Node.js no solo nos permite ejecutar JavaScript fuera del navegador, sino que también nos ofrece un conjunto de herramientas integradas llamadas módulos nativos.

Estos módulos forman parte del propio núcleo de Node y nos permiten realizar tareas muy comunes sin necesidad de instalar nada adicional con npm.

Podemos imaginar los módulos nativos como las “herramientas básicas” de [Node.js](#), listas para usar en cualquier proyecto: leer archivos, trabajar con rutas, crear servidores, manejar eventos, etc.

Por ejemplo (no són los únicos pero sí los más usados?)

- fs (File System): permite leer, crear y modificar archivos del sistema operativo.
- path: ayuda a gestionar rutas y nombres de archivos de forma segura.
- http: sirve para crear servidores web y manejar peticiones.
- os: nos da información sobre el sistema operativo (CPU, memoria, etc.).
- events: permite crear y escuchar eventos personalizados.
- Crypto. encriptar, crear contraseñas seguras, firmar datos o generar tokens.

fs (File System)

Permite leer, escribir, copiar o eliminar archivos en el sistema operativo. Ideal para guardar logs, manejar plantillas o generar archivos dinámicos.

Vamos a crear dos archivos uno que se va a llamar datos.txt con un texto el que queramos y otro que va a leer nuestro archivo usando el módulo nativo fs.

```
JS leerarchivo.js > ...
1  import fs from 'node:fs';
2
3  // Leer el archivo correcto (datos.txt)
4  fs.readFile('datos.txt', 'utf8', (err, datos) => {
5    if (err) {
6      console.error('❌ Error al leer el archivo:', err);
7      return;
8    }
9
10   console.log('📄 Contenido del archivo:\n');
11   console.log(datos);
12 });
13
```

Entendiendo el código. Le dices a Node que quieres usar el módulo nativo fs (File System). Por favor, abre el archivo llamado datos.txt y léelo como texto (utf8). Devuelve (parámetros declarados en la función) una variable que se llama datos o bien devuélveme un error. Si algo sale mal (por ejemplo, el archivo no existe o no tiene permisos), se muestra el error en consola. Si todo fue bien, se muestra el contenido del archivo en pantalla devolverá la variable datos que contiene el texto que había dentro de datos.txt.

Pruébalo en tu Visual Code y analiza con tus palabras.

¿Qué más podemos hacer con el módulo fs?

Leer archivos (fs.readFile)

Escribir nuevos (fs.writeFile)

Copiar, renombrar o borrar archivos (fs.copyFile, fs.unlink, etc.).

Ejercicio.

Intenta crear un archivo; esta vez con fs vamos a crear un archivo nuevo con un texto "Hola soy un archivo nuevo creado desde [Node.js](#)".

Explica cómo se usa el módulo fs

path

Ayuda a trabajar con rutas de archivos y directorios de forma compatible entre sistemas (Windows/Linux). Muy útil al configurar Express o manejar rutas absolutas. El módulo path no se usa directamente en el día a día del desarrollo full stack, pero es fundamental entenderlo, porque Node lo usa constantemente para localizar archivos, carpetas y recursos estáticos.

Vamos a ver un ejemplo de path por ejemplo para buscar un archivo que se va a llamar [path.js](#) y está dentro de una carpeta que se llama api. Creamos un archivo que se va a llamar [buscar.js](#) y lo vamos a crear fuera de la carpeta api.


```
JS buscar.js ×
JS buscar.js > ...
1 // Importamos el módulo nativo 'path'
2 import path from 'node:path';
3
4 // Unimos carpetas y archivo con path.join()
5 // Esto crea una ruta válida en cualquier sistema operativo
6 const rutaArchivo = path.join('api', 'path.txt');
7
8 console.log('✿ Ruta relativa unida:', rutaArchivo);
9
10 // i queremos la ruta ABSOLUTA (completa)
11 const rutaAbsoluta = path.resolve('api', 'path.txt');
12
13 console.log('📍 Ruta absoluta:', rutaAbsoluta);
14
```

Ejercicio.

Tu ejercicio consiste en unir el módulo anterior fs con este nuevo módulo que es tu brújula de archivos. Tendrás que buscar el archivo [path.txt](#) y tienes que escribir en el console.log (“soy un texto nuevo creado con fs”). Crea un nuevo archivo por ejemplo [pathfs.js](#) e intenta que funcione.

Modifica el texto que escribes y vuelve a lanzarlo. ¿Qué ocurre?

Ahora quiero que investigues y pruebes fs.append.

http

El módulo http permite a Node.js crear servidores web que pueden escuchar peticiones del navegador y enviar respuestas.

Gracias a él, tu ordenador puede comportarse como un servidor real:

Cuando alguien entra en una dirección como `http://localhost:3000`, Node recibe la petición y responde.

http es el corazón del backend en Node.js, porque permite la comunicación entre el cliente (frontend) y el servidor (backend).

Ejercicio. ¿Para qué vale el módulo HTTP? Busca los métodos y explica. ¿Qué utilidad piensas que podrían tener?

En una comunicación web intervienen dos partes:

 Servidor (Node.js) Espera y responde a las peticiones.

 Cliente
(navegador) Envía una petición (por ejemplo, al entrar en una URL).

Cuando el cliente envía una petición, Node la recibe como un objeto `req`, y responde con otro objeto llamado `res`.

Ejercicio. Explica qué puertos puedes usar y cuáles son los habituales.

¿Qué es un servidor? Ya hemos visto el concepto servidor y ahora lo vamos a ver funcionando. Un **servidor HTTP** es un programa que escucha peticiones (como cuando visitas una web) y responde con contenido (texto, HTML, JSON...). En Node.js, podemos crear uno desde cero usando el módulo nativo `http`. Un servidor es el que hemos creado en el ejercicio anterior que abre un puerto pero este servidor responde con texto plano.

Crea un archivo y llámalo [servidor.js](#)

Vamos a crear un pequeño servidor.

```
// Archivo: server.js
import http from 'node:http';

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
  res.end('¡Hola desde Node.js! 🚀 \nTu primer servidor está funcionando.');
```

```
});

server.listen(3000, () => {
  console.log('🌐 Servidor ejecutándose en http://localhost:3000');
  console.log('💡 Presiona Ctrl+C para detener el servidor');
```

```
});
```

Analizamos el código.

El módulo `http` es una de las piezas centrales de Node.js.

Permite **crear servidores web** capaces de recibir peticiones del navegador (o de otro cliente) y responder con datos o páginas. El módulo nativo de Node.js se llama `http` y es lo que permite a tu servidor **escuchar** en un puerto (como el 3000) y **entender** las peticiones HTTP que llegan.

Ejemplo mental:

El **cliente** (el navegador) pide:

“Dame la página principal.”

El **servidor** responde:

“Aquí tienes el contenido.”

Crear el servidor

```
const server = http.createServer((req, res) => {

  // ...

});
```

Inma Contreras@2025

El método `createServer()` recibe una función de callback que se ejecuta cada vez que llega una petición HTTP. Luego veremos que significa “callback” te adelanto que es “no te quedes parado y sigue trabajando”.

Cada vez que el servidor recibe una petición, Node llama automáticamente a tu función y le pasa dos objetos especiales:

<code>req</code> (<code>request</code>)	Representa la petición del cliente	Contiene datos como la URL solicitada, el método (GET, POST, etc.), cabeceras, cuerpo, etc.
<code>res</code> (<code>response</code>)	Representa la respuesta del servidor	Permite definir qué se devuelve al cliente: el código de estado (200, 404...), el tipo de contenido y el propio cuerpo de la respuesta.

```
const server = http.createServer((req, res) => {  
  res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });  
  res.end('¡Hola desde Node.js! 🚀 \nTu primer servidor está funcionando.');
```

});

No son palabras reservadas, son nombres elegidos por convención

(son abreviaturas de `request` y `response`).

Tú podrías llamarlos petición y respuesta, aunque casi todo el mundo usa `req` y `res`.

Escuchar en un puerto

Para que el servidor funcione, debemos decirle **en qué puerto escuchará** (el canal de comunicación):

```
server.listen(3000, () => {  
  console.log('🌐 Servidor ejecutándose en http://localhost:3000');
```

console.log('💡 Presiona Ctrl+C para detener el servidor');

});

Inma Contreras@2025

En este primer ejemplo no hemos usado req no nos importa que nos pida el cliente siempre vamos a responder igual pero req podrían ser rutas por ejemplo /contacto /tienda y entonces si toma valores.

Ejercicio. Aquí tienes parte del código. Te aviso que no está completo. Piensa y mira a ver que puede faltar para que funcione.

```
// 2 Analizamos la ruta que el cliente solicita
if (req.url === '/' || req.url === '/inicio') {
  res.statusCode = 200; // código 200 = OK
  res.end('🏠 Bienvenido a la página principal');
}

// 📞 Ruta de contacto
else if (req.url === '/contacto') {
  res.statusCode = 200;
  res.end('📞 Esta es la página de contacto.\nPuedes escribirnos a contacto@nodeejemplo.com');
}

// 🛒 Ruta de tienda
else if (req.url === '/tienda') {
  res.statusCode = 200;
  res.end('🛒 Bienvenido a la tienda online.\nPronto habrá ofertas y productos nuevos.');
```

```
// ❌ Si la ruta no existe
else {
  res.statusCode = 404; // código 404 = No encontrado
  res.end('❌ Página no encontrada. Prueba con /contacto o /tienda');
}
});
```

⌚ timers

El módulo **timers** en Node.js forma parte del núcleo del lenguaje y **permite programar tareas para que se ejecuten después de un tiempo o de forma repetida**. Incluye las funciones setTimeout, setInterval y setImmediate para manejar tareas asíncronas. Lo vamos a ver posteriormente con los callback pues es una forma de simular callback.

API en Node.js

Para que el front se comuniquen con el back hace falta crear una API. Una API permite que dos programas se comuniquen. La api responderá con los datos que se soliciten. En el desarrollo web, una API es un **servidor que responde con datos** (normalmente en formato JSON) cuando el frontend hace una petición.

API son las siglas de Application Programming Interface, que en español significa Interfaz de Programación de Aplicaciones.

Una API en Node.js es un servidor que:

- Escucha peticiones HTTP (GET, POST, etc.)
- Analiza la URL (/api/saludo, /api/hora)
- Devuelve datos en formato JSON
- Usa códigos de estado HTTP (200, 404, etc.)

Todas las apps modernas usan APIs para comunicarse con el backend. Permite crear tu propio servidor de datos. Frameworks como Express.js se basan en este patrón para simplificar el desarrollo.

Crea esta pequeña Api

```
import http from 'node:http';

const server = http.createServer((req, res) => {

  // Cabeceras: CORS y tipo de respuesta JSON

  res.setHeader('Access-Control-Allow-Origin', '*');

  res.setHeader('Content-Type', 'application/json; charset=utf-8');
```

```
// GET /api/saludo

if (req.url === '/api/saludo' && req.method === 'GET') {

  const data = { ok: true, mensaje: 'Hola desde Node API 🍌', modulo:
'UF1844' };

  res.writeHead(200);

  return res.end(JSON.stringify(data)); // objeto JS → texto JSON
}


// GET /api/hora

if (req.url === '/api/hora' && req.method === 'GET') {

  const ahora = new Date();

  const hh = String(ahora.getHours()).padStart(2, '0');

  const mm = String(ahora.getMinutes()).padStart(2, '0');

  res.writeHead(200);

  return res.end(JSON.stringify({ ok: true, hora: ` ${hh}:${mm}` }));
}


// Cualquier otra ruta

res.writeHead(404);

res.end(JSON.stringify({ ok: false, error: 'No encontrado' }));
});


// Puerto 3000

server.listen(3000, () => {
```

Inma Contreras@2025

```
console.log('✅ API en http://localhost:3000 (GET /api/saludo, /api/hora)');  
});
```

¿Qué acabamos de crear?

En este ejemplo se construye una **API muy simple** utilizando sólo el **módulo nativo http** de Node.js. Esto significa que **no se usa ningún framework externo** como Express: todo el flujo de la petición y la respuesta se controla directamente con las herramientas que Node proporciona de forma predeterminada.

Ejercicio.

¿Qué diferencias ves entre este código que crea una Api y el anterior de las rutas?

Ahora te toca a ti.

Crea una API simple con Node.js que devuelva mensajes diferentes en formato JSON según la ruta que el cliente pida. Lo vamos a llamar [miapi.js](#)

Crea dos rutas. Ánimo lo hemos visto todo.

Fetch(): Comunicación entre el Frontend y el Servidor

¿Qué es `fetch()`?

`fetch()` es una función nativa de JavaScript (no necesitas instalar nada) que permite al navegador conectarse con un servidor y pedir o enviar información.

En palabras sencillas: `fetch()` es el puente que conecta tu frontend (HTML, JS, React...) con tu backend (Node, Express, APIs, bases de datos...).

Se usa para realizar peticiones HTTP (como GET, POST, PUT, DELETE) a una dirección (URL), y recibir la respuesta del servidor.

- Es una función del navegador para hacer peticiones HTTP (GET, POST...).
- Devuelve una promesa con un objeto Response.
- Para leer los datos JSON: `await res.json()`.

Sintaxis básica

```
js

fetch(url, opciones)
  .then(respuesta => respuesta.json())
  .then(datos => {
    // trabajar con los datos del servidor
  })
  .catch(error => console.error('Error:', error));
```

Parámetros:

Parámetro	Descripción
<code>url</code>	La dirección a la que se hace la petición (por ejemplo, tu API en Node o Express).
<code>opciones</code>	Un objeto opcional donde indicas el método (<code>GET</code> , <code>POST</code> , etc.), cabeceras o el cuerpo (body) con los datos que envías.

`fetch()` también sirve para **enviar datos** desde el frontend al backend, por ejemplo, en un formulario o registro de usuario.

 Ciclo completo de `fetch`

```

java
graph TD
    subgraph FRONTEND ["FRONTEND (navegador)"]
        direction TB
        A[fetch() → petición HTTP GET/POST...]
    end
    A --> B["BACKEND (Node / Express)"]
    subgraph BACKEND ["BACKEND (Node / Express)"]
        direction TB
        C[res.json() → respuesta JSON]
    end
    C --> D["FRONTEND (navegador)"]
  
```

- 1 El **frontend** hace la petición con `fetch()`.
- 2 El **backend** la recibe, la procesa y responde con datos JSON.
- 3 El **frontend** convierte la respuesta con `.json()` y la usa en su interfaz.

El método `.json()`

Cuando usas `fetch()`, la respuesta del servidor **no se convierte automáticamente** en un objeto JavaScript.

Primero debes transformarla:

```
fetch('/api/saludo')
```

`.then(res => res.json())` // transforma el texto JSON → objeto JS

`.then(data => console.log(data));`

Siempre es buena práctica añadir un `.catch()` para capturar posibles errores de red o de servidor

```
fetch('/api/hora')
  .then(res => {
    if (!res.ok) throw new Error('Error en la respuesta del servidor');
    return res.json();
  })
  .then(data => console.log('Hora actual:', data))
  .catch(error => console.error('Hubo un problema:', error.message));
```

Vamos a verlo en un ejemplo muy sencillo.

Crea un front con un botón. Y vamos a crear un script que use fetch. Estamos en el front. Algo parecido a esto. Lo importante es que tengas un botón. .

`<!DOCTYPE html>`

`<html lang="es">`

`<head>`

`<meta charset="UTF-8" />`

`<title>Ejemplo fetch básico</title>`

`</head>`

`<body>`

`<h1>Ejemplo: fetch desde el navegador</h1>`

`<button id="boton">Cargar saludo</button>`

`<p id="resultado"></p>`

Inma Contreras@2025

```
<script>

// Cuando el usuario hace clic en el botón

document.getElementById("boton").addEventListener("click", () => {

  // 1 Hacemos la petición al servidor. Aquí es donde introducimos Fetch.

  // estamos en el front

  fetch("http://localhost:3000/api/saludo")

  // 2 Convertimos la respuesta a JSON

  .then((res) => res.json())

  // 3 Mostramos los datos en la página

  .then((data) => {

    document.getElementById("resultado").textContent =

      data.mensaje + " — " + data.curso;

  })

  // 4 Capturamos errores (por si el servidor no responde)

  .catch((err) => console.error("Error:", err));

});

</script>

</body>

</html>
```

y ahora crea el servidor

```
// Servidor con una sola ruta

const server = http.createServer((req, res) => {

  // Permitir CORS y definir formato JSON

  res.setHeader('Access-Control-Allow-Origin', '*');

  res.setHeader('Content-Type', 'application/json; charset=utf-8');


  // Si la ruta es /api/saludo

  if (req.url === '/api/saludo' && req.method === 'GET') {

    const respuesta = { mensaje: '¡Hola desde Node.js 🙌!', curso: 'UF1844' };

    res.writeHead(200);

    res.end(JSON.stringify(respuesta)); // enviamos texto JSON

  }

});


server.listen(3000, () => {

  console.log('✅ Servidor en http://localhost:3000');

});
```

Analiza que hemos hecho. ¿Cual ha sido la conexión entre el front y el back?
¿Cómo sabe el back que tiene que devolver? ¿Qué errores te ha dado?.



Ejercicio. Te toca a ti. Conecta la api que hiciste antes de las dos rutas con un front.

El paquete NPM.

npm significa Node Package Manager (Gestor de Paquetes de Node).

Es un programa que viene instalado automáticamente cuando instalas Node.js, y cumple dos funciones principales:

1. Registro de Software (La Tienda Online): Es una base de datos gigantesca y pública en Internet (el "Registro npm") donde los desarrolladores suben y comparten su código como paquetes (también llamados módulos o librerías).

2. Herramienta de Línea de Comandos (El Carrito de Compras): Es el comando que usas en tu terminal (npm install...) para descargar ese código compartido e instalarlo en tu proyecto.

Su función principal es instalar, actualizar, compartir y administrar librerías (módulos) que amplían las capacidades de tus proyectos Node. NPM es como una tienda de herramientas donde los desarrolladores de todo el mundo publican módulos listos para usar.

npm se encarga de tres tareas fundamentales en tu desarrollo:

1. Instalar Dependencias (npm install)

Esta es la función más común. Te permite añadir código de terceros a tu proyecto para no tener que programar todo desde cero.

2. Gestionar el Proyecto (package.json)

Cada proyecto de Node.js debe tener un archivo central llamado package.json. npm te ayuda a crearlo y mantenerlo.

- ¿Qué es? Es un archivo de configuración en formato JSON que actúa como el carnet de identidad de tu proyecto.
- ¿Qué contiene?
 - El nombre y la versión de tu proyecto.
 - Los scripts (atajos) para ejecutar tu aplicación (ej: npm start).
 - La lista de dependencias (todos los paquetes que tu proyecto necesita para funcionar).

Ejecutar Tareas (npm run)

npm te permite definir comandos personalizados en tu package.json para automatizar tareas repetitivas, como iniciar el servidor.